NLP - The Ghost in the Machine

Monishram Selvaraj

This project is an investigation on whether authorship itself, independent of topic, leaves a measurable statistical/linguistic fingerprint in the text. Using three datasets - human, generic AI, and stylised AI (ordering the model to mimic a particular author's style) - the study analyses different features of the text, namely, lexical, semantic, syntactic, etc. and establishes whether the three differ statistically before any training is involved. Three types of detectors are implemented, in logically increasing accuracy. Beyond the detection, an analysis is done on the interpretability of the models - what factors the different models look for to determine the class of a particular text. Finally, a generative algorithm was developed to see if AI text can be iteratively evolved to bypass these models.

All tasks were completed except Task 3: Error Analysis

Task 0 - The Library of Babel

The goal of Task 0 was to construct a clean dataset where the primary variable is authorship, and style, not topics. To achieve this, 10 novels from only two authors (D.H. Lawrence, and H.P. Lovecraft) were downloaded from Project Gutenberg. The downloaded text files were stored in data/raw/.

0.1 Cleaning

There are various unwanted data in a Gutenberg text file, like TOC, page numbers, image extensions (in a UTF-8 encoded file), but a common characteristic of Gutenberg books helped a lot in cleaning the data. Every prose starts and ends with a marker:

*** START ***

*** END ***

Using this, subtexts like the preface, copyright descriptions, etc. were easily removed. Even in this filtered text, empty lines, lines just with dashes, chapter names, and other insignificant subtexts were still present. Empty lines, lines with "CHAPTER" included, which were dash-heavy, which had all uppercase characters, and if they included any of ".png"/".jpg"/".jpeg" were filtered out and removed from the dataset.

0.2 Chunking

This was carried out by checking where a sentence ended, after having atleast 500 words. Each chunk was stored in a separate text file in data/human/.

0.3 Topic Extraction

To extract "topic words" from each chunk, checking the most occurring word was a flawed method, since this would probably give common words like prepositions, conjunctions, etc. This was solved using a concept called TF-IDF (Term Frequency - Inverse Document Frequency) - checking the "importance of a word in a subgroup of data". The algorithm fit perfectly, because each chunk would be considered a subgroup of the data. There are two parts to this method:

TF: the frequency of each word was recorded and normalised - Number of times a term appears/Total terms in the document

IDF: measures how important a term is across all the documents -

log (Total number of documents in the dataset/Number of documents containing the term)

TF-IDF value is simply the product of both these terms - TF x IDF

WRITE DOWN THE LATEX EXPRESSIONS FOR THIS

Essentially, if a word has a high frequency in more than one document, its importance goes down in all of them.

The TF-IDF value of all the words were calculated, and the top 10 words were taken per chunk. These topic words were stored in a file in data/topics/.

0.4 Paragraph Generation

A Gemini API was used to generate 50 paragraphs per topic, 25 each for a generic prompt and a stylised prompt, where the model is told to mimic their style. These paragraphs were then stored in data/generic_ai/ and data/stylized_ai/.

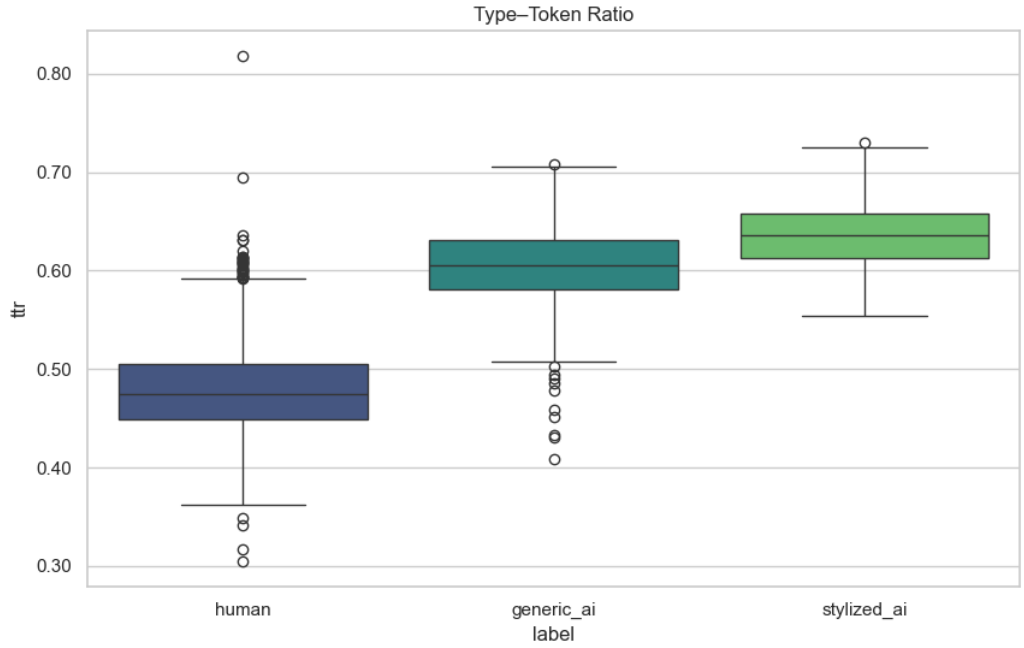
Since generating such a huge number of paragraphs took a really long time, a small improvement was made by making parallel requests, using a simple thread pool executor. This resulted in the execution time almost halving, sometimes even reducing by 60-65%.

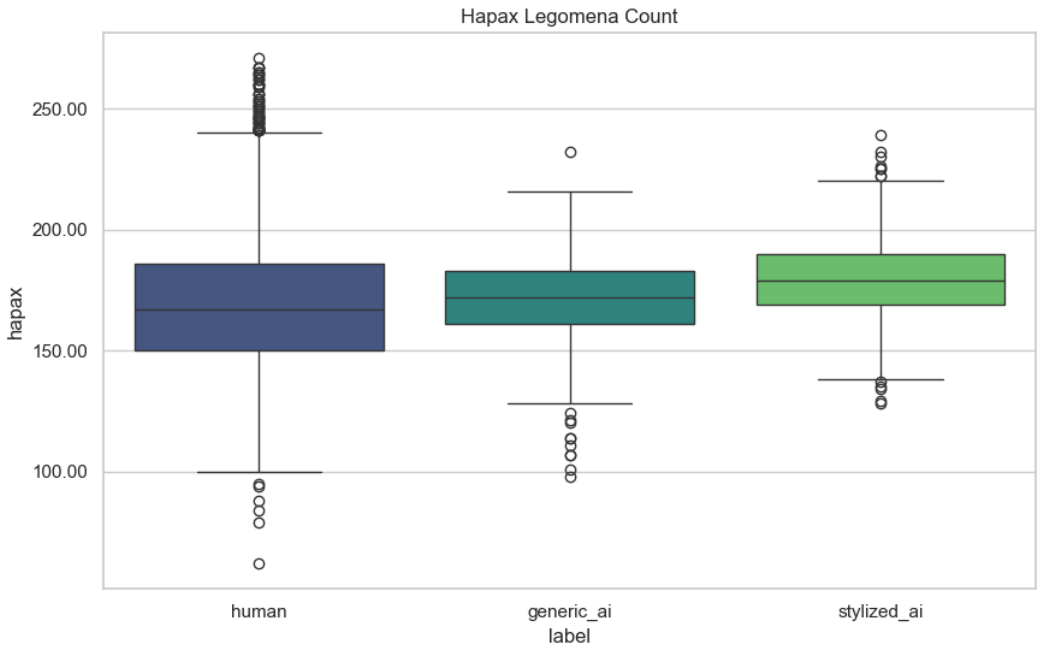
ThreadPoolExecutor creates a fixed number of worker threads (max_workers=5), where each thread can run a function independently while the main program continues scheduling tasks.

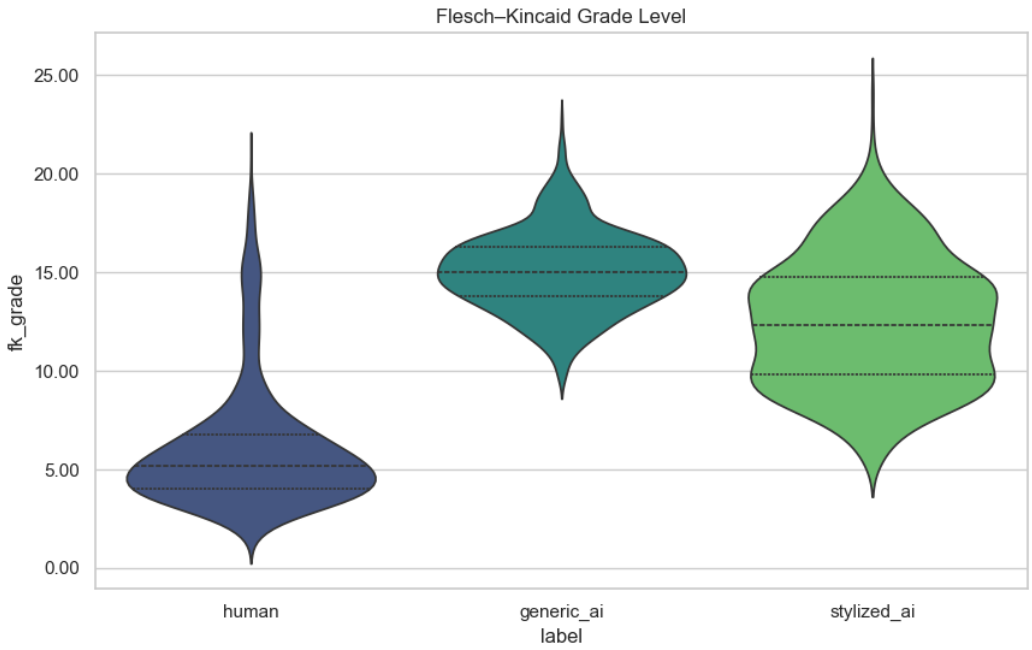
Task 1 - The Fingerprint

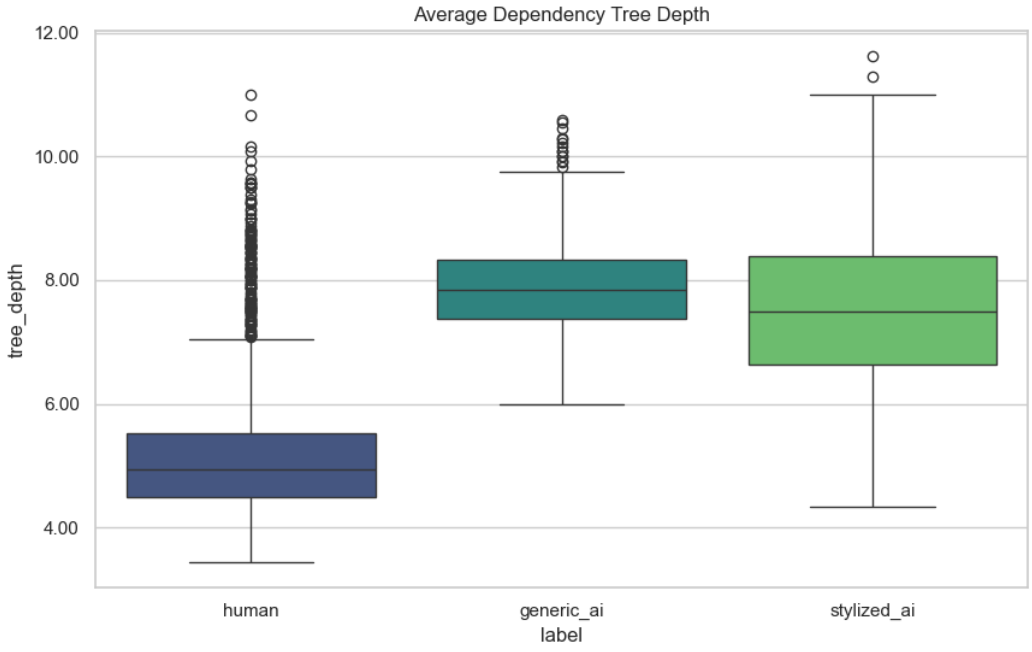
We were meant to measure the values of different metrics in this task, which are as follows:

- Type-Token Ratio (TTR) - Unique words/Total words (Unique words ≠ Words with frequency=1)
- Hapax Legomena - Number of words occurring only once in a sample.
- Part of Speech (POS) Distribution - Number of adjectives/Number of nouns
- Dependency Tree Depth - Each sentence can be represented as a tree, where there is a fixed mapping between the different parts of a sentence (nouns, verbs, adjectives, etc.). The deeper the tree is, the more complex the sentence is.
- Punctuation Density - a normalised count of the frequency of different types of punctuation
- Flesch-Kincaid Grade Level - WRITE THE PROPER DEFN HERE


Type–Token Ratio
ttr
0.80
0.70
0.60
0.50
0.40
0.30
human
generic_ai
stylized_ai
label


Hapax Legomena Count
hapax
250.00
200.00
150.00
100.00
human
generic_ai
stylized_ai
label

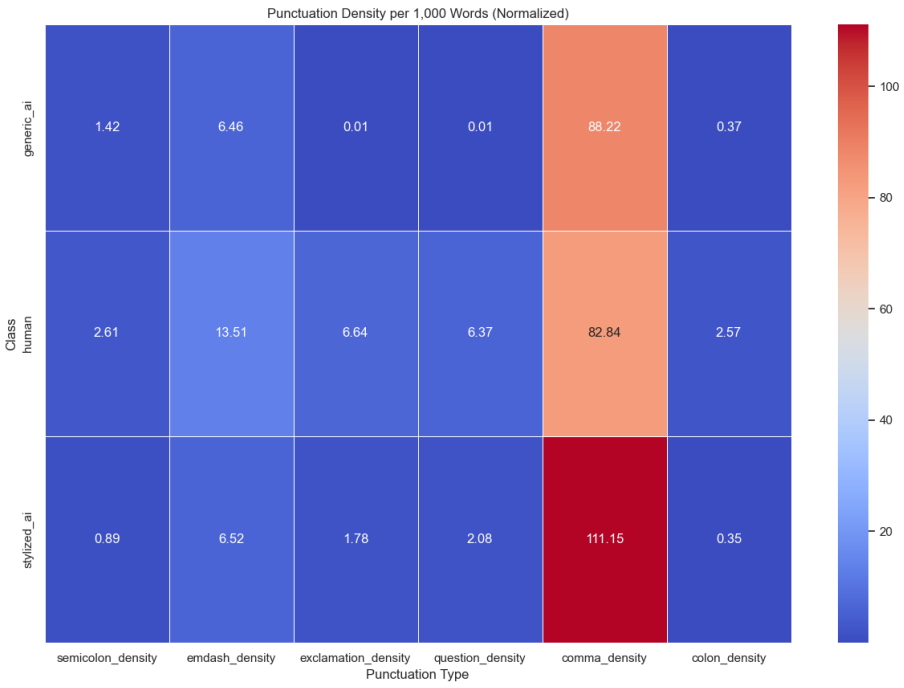

Flesch–Kincaid Grade Level
fk_grade
25.00
20.00
15.00
10.00
5.00
0.00
human
generic_ai
stylized_ai
label


Average Dependency Tree Depth
tree_depth
12.00
10.00
8.00
6.00
4.00
human
generic_ai
stylized_ai
label

Observing these plots, the average of the measurements of each metric is plotted below:

Metric/Class	Human	Generic AI	Stylised AI
TTR	0.47	0.61	0.63
Hapax Legomena	166.8	171.2	178.2
POS Distribution	0.487	0.429	0.508
Flesch-Kincaid Grade	5.3	15.01	12.3
Dependency Tree Depth	5.02	7.76	7.24

These metrics are the basic differentiators between the three classes; as we can see, in most cases, stylised AI is closer to human values than generic AI, which partly proves that providing a prompt that mentions the author's style is improving the human style of a paragraph.



The heatmap confirms that stylised AI is not simply a better version of generic AI but has a distinct syntactic fingerprint characterised by comma overuse (111.15/1k words). Meanwhile, human writing is distinguished not by 'perfect' grammar, but by structural complexity - utilising em-dashes, semicolons, and rhetorical punctuation that AI models avoid.

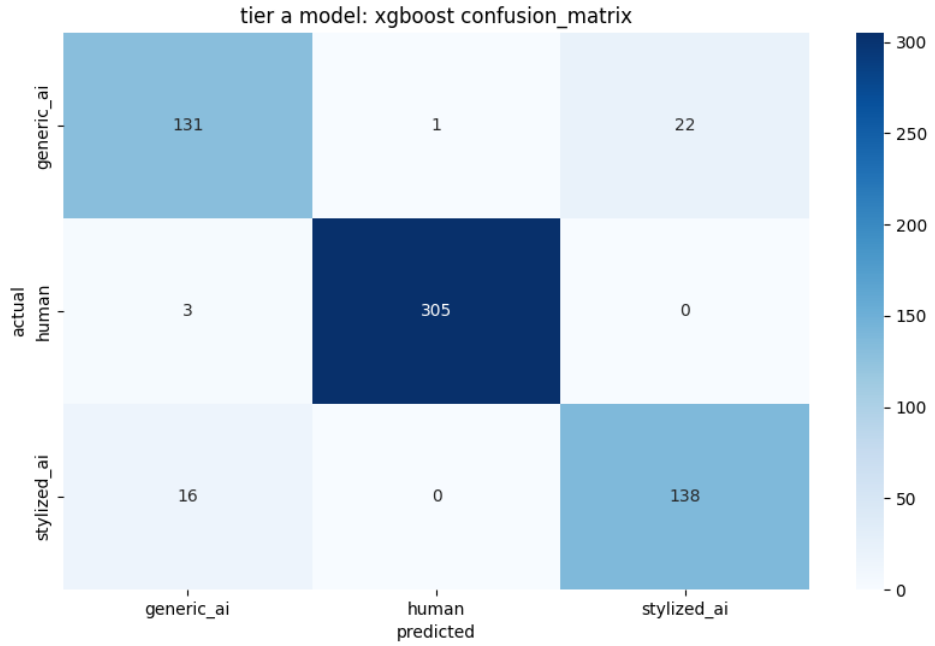
Task 2 - The Multi-Tiered Detective

2.A - The Statistician

This model is trained on the statistical values we measured in Task 1. It runs on XGBoost (eXtreme Gradient Boosting), which is an algorithm that builds a classifier by combining multiple decision trees. Each new tree is trained to correct the errors made by the previous ones, by using a loss function that it minimises. XGBoost was chosen over Random Forest since the latter does not have the capability to learn from the previous decision trees' mistakes.

The train-test ratio is taken to be 80:20, and initially, the model gave a surprisingly high accuracy of **99.89%** when tested with the old literature books. This was occurring because the old literature books stood out in the measurements of the parameters; unique words were more, the sentences were more descriptive (higher POS distribution), etc. After changing the dataset to a more modern writing style, the accuracy decreased to **93.18%**.

As shown below in the confusion matrix, the human paragraphs were still very easy to detect, which still points to the somewhat old literature style being used. Upon research, the authors whose books were used, were still not following modern (1920s) style, but were definitely not following the literature used by Shakespeare; for example, words like "thou", "thy", "prithee" were eliminated from the dataset, which improved the training of the model.

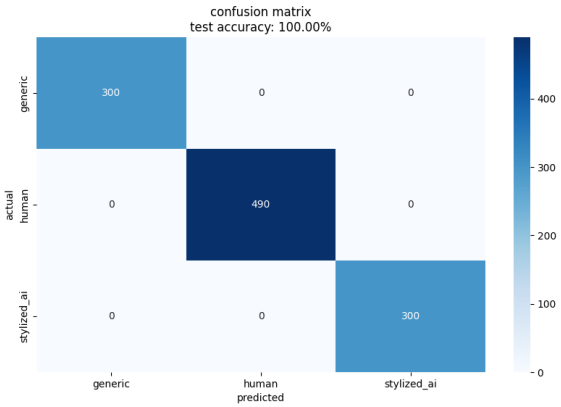


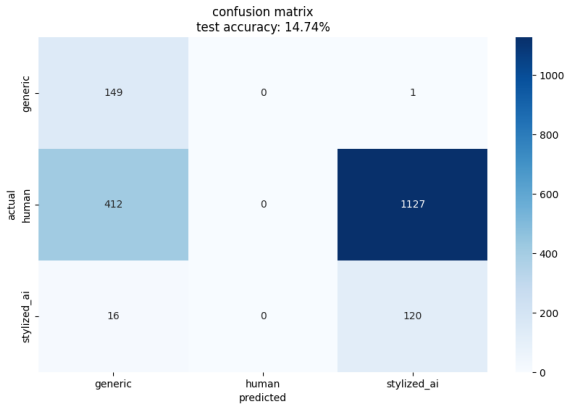
2.B - The Semanticist

The model implemented here, used a feedforward neural network, using averaged pre-trained embeddings (provided by GloVe - Global Vectors for Word Representation).

The GloVe file used in this implementation has around 400,000 words' embeddings, which are used to find the average vector of a sentence, and a paragraph henceforth. Using the vector for a paragraph, the neural network is trained to recognise the authorship.

The train-test ratio is taken to be 80:20 too, and the model gave a surprisingly high accuracy of **100%**. This was suspected to be because all the generated chunks topic-wise were stored into their respective directories one after the other, which was then retrieved by the model in an organised way - human writing chunks, generic AI chunks, then stylised AI chunks. So, the model had been training on the human set and part of the generic AI set, and testing on the remaining of the generic and stylised AI sets. This was fairly easy even for a human to distinguish, all the model had to do is check if the paragraph aligns with the topic that the human writing was originally based on. To solve this, the dataset was shuffled before the train-test split. The accuracy then dropped to **14.74%**, which was way too low for a 3 layer neural network.





Since GloVe is a pre-trained embedding system, it will only provided fixed vectors for a particular word. Why can't we use the meaning of a word with respect to its context, to get a more optimised vector? To achieve this improvement, I implemented a Gemini embedding model, which is optimised for SEMANTIC SIMILARITY, which understands the context and provides embeddings accordingly.

For example, the two sentences - "River banks are very fertile" and "Depositing money in banks is very safe", both contain the word "banks", but both of them have a different meaning. The implemented model will understand this, and give different embeddings for both of them. GloVe on the other hand, will give the same vector for both occurrences of " banks".

To store the vectors generated, a ChromaDB vector database was created, for quick writing and retrieval. Instead of storing the vectors directly, and just reading from a file, the choice of a vector database is much more advantageous since it supports low-latency retrieval, and efficient indexing for high dimensional vectors, which is required since text-embedding-001 (model for embedding in Gemini) creates 768 numerical values for each embedding. Doing this in a file-based storage system is a great hassle and would slow the process down immensely.

2.C - The Transformer

This model was a deep learning model, which used dilbert-base-uncased and LoRA (Low Rank Adaptation).

- LoRA is a parameter efficient fine tuning (PEFT) technique that enables large models with a very high amount of parameters to be used for small tasks. It freezes the main model, and inserts a matrix of modifiable parameters in between the layers, so it doesn't disturb the main layout of the model.

- BERT is a transformer model that learns word meaning using context from both directions in a sentence, making it effective for classification tasks. DistilBERT is a smaller, faster version of BERT that retains most of its accuracy while being more computationally efficient, which made it suitable for this task.

- DilBERT was chosen over RoBERTa because the overall efficiency of DilBERT is better; it's more of a clone of the main model BERT, while RoBERTa is a smaller version, which has faster computing, but lower accuracy.

Observations:

Initially, the accuracy was very high (>98%):

Epoch	Training Loss	Validation Loss	Accuracy	F1	Precision	Recall
1	No log	0.036089	0.986068	0.986066	0.986090	0.986068
2	No log	0.036459	0.984520	0.984457	0.985019	0.984520
3	No log	0.015610	0.996904	0.996910	0.996945	0.996904
4	No log	0.013301	0.996904	0.996910	0.996945	0.996904
5	No log	0.019436	0.992260	0.992264	0.992324	0.992260

The main suspicion was just the max token length being 64. To improve the output, the max token length was changed to 256, which gave a negligible difference in accuracy:

Epoch	Training Loss	Validation Loss	Accuracy	F1	Precision	Recall
1	No log	0.088948	0.975232	0.975655	0.977634	0.975232
2	No log	0.050460	0.987616	0.987719	0.988247	0.987616
3	No log	0.033428	0.992260	0.992295	0.992511	0.992260
4	No log	0.017486	0.995356	0.995369	0.995448	0.995356
5	No log	0.017084	0.995356	0.995369	0.995448	0.995356

Another big suspicion was the genre of text being used. Since the downloaded files from Project Gutenberg were in the style of old literature, and mainly novels and proses, the model would've thought that anything informative is AI, and anything in a prose's style is written by a human. This was confirmed with a small text, where a sample of a novel generated by a prompt was given to the model, and it incorrectly guessed human.

INSERT RESULTS

So, to confirm this, more modern text data was fed to the model, and it improved the model by a significant amount. In addition, the model was changed to DeBERTa, which was chosen since it's currently the best BERT model (Microsoft).

Conclusion:

The first model was failing due to genre overfitting, something occurring when a model is fed texts from the same topics, in that particular class. This leads to the model guessing based on the topic of the text, and not on the lexical, syntactical values. Hence, this is flawed and highly inaccurate, because even if a text is fed which doesn't come under that topic, the model will look at it and simply guess the other class.

The second, improved model, however, learnt from various different genres/topics, so the guesses it makes are based more on a general idea - topic-based, as well as other factors like vocabulary, lexical richness, etc.

Other Fine Tuning techniques which were attempted:

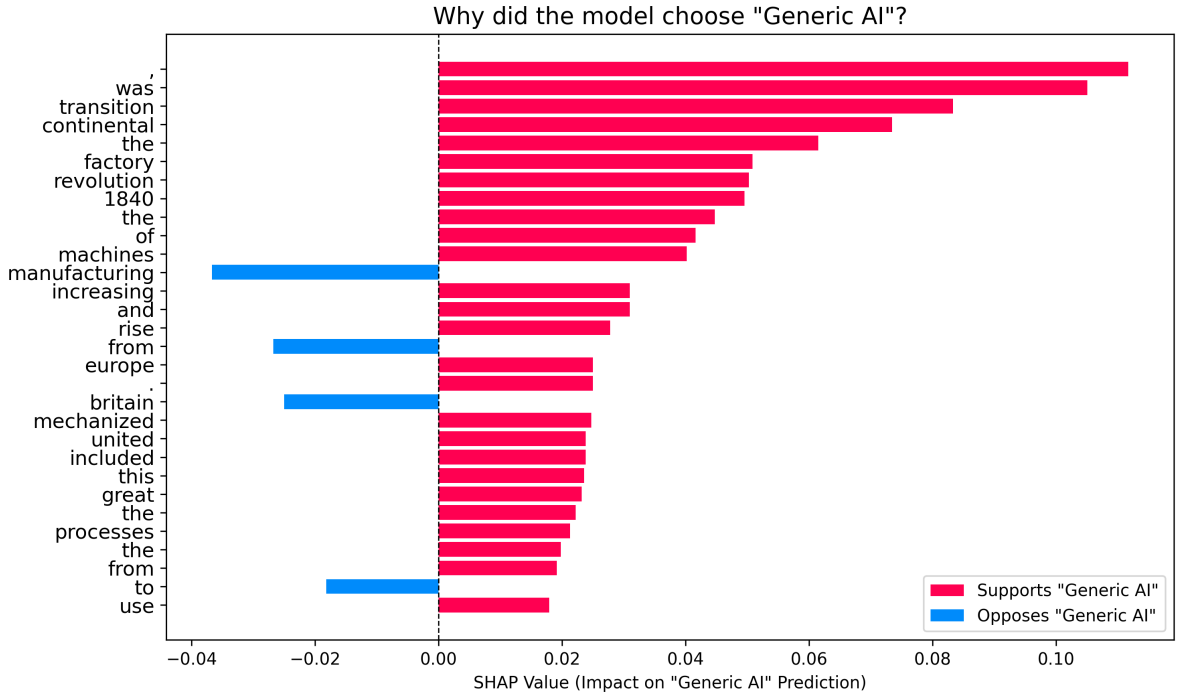
- QLoRA is a quantised version of LoRA, which applies quantisation to the base model, reducing memory usage further. In this case, the benefit was limited since DistilBERT is already lightweight, and standard LoRA performed similarly.
- Prefix tuning freezes the entire model and learns a small set of trainable tokens that guide the model's behaviour. For example, before prompting something to the model, we feed it some prefix tokens - "Imagine you are an NLP researcher who has written multiple A* papers". This would change the output of the model, and make it more accurate. Though, it wasn't possible to implement it here since the BERT models didn't allow it.

Task 3 - The Smoking Gun

This task represents the observations made from the predictions made by Tier C model (Deep Learning) from Task 2. We make them by studying how the model caught an AI generated paragraph.

We use SHAP (SHapley Additive exPlanations), which is a library that is used to figure out which particular words influenced the model to guess AI and not human. Each word is assigned a ”SHAP score”, who’s magnitude represents how much it influences the decision, positive if supporting the decision and negative if opposing.

The Tier C model is a Deep Learning model than distinguishes between human, generic AI and stylised AI writings. To understand how the model actually classifies a paragraph, SHAP is utilised.



The above plot is the proof for the ”genre mismatch” that’s happening; it clearly shows that the model is not looking for AI hints, rather it’s looking for ”textbook” style v/s ”story” style. Words like "transition", "continental", "period", "increasing", and "mechanized" are all red, meaning they signal AI strongly. These are more descriptive words than just adjectives, which is evidence that the model thinks abstract/conceptual words = AI.

Note that the comma (,) is the strongest signal that the text is AI, which might imply that any ”Oxford” listing method, is a very key indicator that a writing is AI generated to this model.

On the other hand, ”Britain” and ”manufacturing” are two words that oppose the generic AI decision, which means that concrete nouns, and locations, or proper nouns in general, are indicative of human writing.

Conclusion:

The saliency mapping done is visual proof of how the model thinks, and it perfectly matches the assumptions made in the previous task, about how the data fed into the model affected its decisions, and how vague it is based on the factors (words) affecting its decisions. It follows a very heavy syntactic and genre bias, where it takes in factors like the comma, and nouns used in a way to determine the class, distinguishing between descriptive text and narrative fiction.

Task 4 - The Turing Test

4.1 The Super Imposter

Can a paragraph be evolved to bypass the best model? Can we implement a genetic algorithm to optimise a piece of AI-generated text such that it gets flagged as human written?

This task is done in accordance with the statement of the previous task, to use either a distilbert-base-uncased or roberta-base model, with LoRA fine tuning. The best model so far was DeBERTA, but that was not used. Also, only slightly modern/old literature from Project Gutenberg was used, as mentioned in Task 0. The

According to our hypothesis earlier, the model learnt based on a very particular type of writing, which had two notable characteristics:

- Semantic features like old vocabulary, sentence framing, among others that AI generated text doesn't normally have.
- Narrative, prose-like writing, which don't have any statements; not being factual, informative.

Hence, evolving a paragraph is merely just changing the mentioned features to align with the provided human dataset.

Working of the GA:

- Initially, an AI generated text is given to the model, which then gives its prediction, and human probability score, or the "fitness function". If the score is >50%, the paragraph needs a drastic change, else it's close, so minor polishing is done. 3 paragraphs are to be generated per generation, and all three are again tested, out of which the best one is chosen. This is then the parent of the next generation. This came out with a human confidence score of 99.14%
- DistilBERT reads the text in tokens, and AI writing has highly probable tokens, while human writing has unpredictable, spiky tokens. To increase the confidence even more, minor tweaks were taken care of:
  - Punctuation: semicolons, em-dashes, are considered risky and complex, not usually used in AI generated text
  - Proper Nouns: AI is usually vague in specifying nouns, while humans are specific, more often naming the noun.

Our algorithm evidently didn't need much generations to evolve the paragraph, since the model is very poorly trained. The results prove our hypothesis, i.e., all it took for the AI to evolve the paragraph was add some narrative style, and a 99.14% confident human prediction was made in the first generation itself.

4.2 The Personal Test

Feeding the SOP in the classifier model, I get a very high generic AI prediction, with all paragraphs coming out to be >99% confidence. This aligns well with the hypothesis that we made earlier, that the model thinks human writing is just narrative. The SOP fed, was more of a formal, informative paragraph, hence also confirming the hypothesis and the model giving a valid prediction.

The major step to customising this paragraph manually to get a human prediction is to make the text more narrative and add a prose-like style to it. This was achieved by adding phrases like "I usually notice it at the beginning, before anything looks like progress" and "I've learned to stay with that discomfort rather than rush past it", which successfully increased the human probability score.

Attached below are three tests, first one being a single paragraph of the SOP, and the second and third being the modified versions. Although not by much, the human probability score increased a little, for small changes such as previously stated.

```
input paragraph I usually notice it at the beginning, before anything look

prediction Generic AI
confidence    99.99411010742188
human         0.00512375682592392
generic ai    99.99411010742188
stylized ai   0.0007690081256441772


input paragraph Most problems don't arrive loudly. They slip in looking fi

prediction Generic AI
confidence    99.82060241699219
human         0.11480676382780075
generic ai    99.82060241699219
stylized ai   0.06459309160709381


input paragraph Most problems don't arrive loudly. They slip in looking fi

prediction Generic AI
confidence    98.71785736083984
human         1.2775253057479858
generic ai    98.71785736083984
stylized ai   0.004623054526746273
```